Taller Práctico Integrador

"Adivina la Palabra Secreta / Ahorcado de Consola" en Java

1. Introducción y Configuración del Proyecto en IntelliJ IDEA

El propósito de este taller práctico integrador es articular y consolidar las competencias adquiridas en las Unidades 1.1 a 1.4 del trayecto formativo de desarrollo en Java (*Java Developer Roadmap*). A través de un enfoque pedagógico orientado al aprendizaje activo, el estudiante asumirá el rol de desarrollador principal para construir una aplicación interactiva de consola: el juego **"Adivina la Palabra Secreta / Ahorcado"**.

Este proyecto no solo requiere la aplicación de sintaxis básica, sino también el diseño estructurado de flujos de control, la gestión eficiente de la memoria mediante tipos primitivos y objetos, el manejo dinámico de texto y la construcción de interfaces legibles por consola.

Para dar inicio al desarrollo, el estudiante debe configurar su entorno de trabajo en **IntelliJ IDEA** siguiendo la guía estandarizada de creación de proyectos:

1. **Creación del Proyecto:** Abrir IntelliJ IDEA, seleccionar *New Project*, nombrar el proyecto como `AhorcadoConsola` y asegurar la selección del JDK de Java correspondiente.
2. **Estructura de Paquetes:** Crear la estructura de paquetes dentro de la carpeta `src/`, utilizando la convención estándar `com.educacion.ahorcado`.
3. **Clase Principal:** Crear el archivo de la clase principal nombrado `Main.java` dentro del paquete estructurado, el cual albergará el método de entrada `public static void main(String[] args)`.

Resumen Visor de Unidades

Unidad del Módulo	Aplicación Práctica en el Juego
Unidad 1.1: Fundamentos y Tipos de Datos	Definición de variables de estado, constantes inmutables (`MAX_INTENTOS`), acumuladores de puntaje y tipos primitivos.
Unidad 1.2: Estructuras Seleccionadoras	Implementación de menús dinámicos con `switch` y validación condicional de opciones.
Unidad 1.3: Bucles y Control de Flujo	Iteración de turnos mediante `while` / `do-while`, control de saltos con `break` / `continue` y repetición del juego.
Unidad 1.4: Manejo de Cadenas y Memoria	Normalización de cadenas (`String`) y manipulación dinámicamente eficiente de la máscara de texto con `StringBuilder`.

📌 **Paso Siguiente:** Una vez configurado el entorno de trabajo en IntelliJ IDEA y comprendida la integración modular del proyecto, es momento de diseñar la arquitectura del menú principal y el control de flujo de la aplicación.

2. Fase 1: Arquitectura del Menú Principal y Control de Flujo (Unidades 1.2 y 1.3)

La primera fase del desarrollo consiste en construir la infraestructura de navegación de la aplicación. Para garantizar que el programa se mantenga en ejecución continua hasta que el usuario decida finalizarlo explícitamente, el menú debe estar envuelto en un ciclo repetitivo de tipo `do-while`.

Especificaciones del Menú Interactivo

El sistema debe desplegar un menú de opciones estructurado de la siguiente forma utilizando una sentencia seleccionadora `switch`:

- `1. Jugar`
- `2. Ver Estadísticas`
- `3. Salir`

Control y Validación de Entradas

Para evitar fallos de ejecución al procesar datos no válidos, el estudiante debe implementar un mecanismo de validación repetitiva apoyado en un bucle `while`. La lectura de datos de consola debe procesar cadenas de texto y transformarlas numéricamente utilizando el método `Integer.parseInt()` o clases de envoltorio (*Wrapper*). Si el usuario ingresa un valor alfanumérico no permitido o una opción fuera del rango (diferente de 1, 2 o 3), el sistema debe capturar la eventualidad, mostrar un aviso pedagógico de error y reintentar la lectura sin romper la ejecución del programa.

💡 **Consejo del Mentor:** Al leer datos por consola con `Scanner`, priorice leer siempre líneas completas con `nextLine()` y parsear el valor a entero mediante `Integer.parseInt()`. Esto evita los problemas habituales de caracteres residuales en el búfer de entrada que ocurren al alternar entre métodos numéricos y de texto.

📌 **Paso Siguiente:** Teniendo estructurado el menú interactivo y garantizado el control de entradas, el siguiente paso consiste en registrar la información del jugador e inicializar las variables de estado del sistema.

3. Fase 2: Registro de Jugador e Inicialización de Datos (Unidades 1.1 y 1.4)

Antes de dar inicio al bucle de la partida, el programa debe solicitar el nombre del usuario y preparar las variables que controlarán la lógica interna del juego. La adecuada elección de tipos de datos y el uso de constantes aseguran la integridad y el rendimiento de la aplicación.

Tabla de Variables y Tipos de Datos

Nombre de Variable	Tipo de Dato	Propósito	Ejemplo de Valor
`MAX_INTENTOS`	`final int`	Constante inmutable que fija el límite superior de fallos permitidos.	`6`
`intentosRestantes`	`int`	Variable contadora que decrementa con cada fallo del jugador.	`6`
`puntaje`	`double`	Registro decimal acumulativo del rendimiento obtenido.	`100.0`
`juegoGanado`	`boolean`	Bandera (*flag*) de estado para verificar la condición de victoria.	`false`
`ultimaLetra`	`char`	Almacenamiento temporal del último carácter ingresado.	`'A'`
`nombreJugador`	`String`	Cadena para almacenar la identificación del usuario normalizada.	`"CARLOS"`

Requerimientos de Normalización de Texto

El nombre ingresado por el usuario debe someterse a un proceso estricto de saneamiento utilizando los métodos de la clase `String`:

1. **Limpieza de espacios:** Utilizar `.trim()` para eliminar espacios en blanco al inicio o al final de la entrada.
2. **Estandarización:** Convertir el texto a mayúsculas sostenidas mediante `.toUpperCase()` para evitar inconsistencias de formato.
3. **Validación de contenido:** Evaluar la longitud con `.length()` para asegurar que el usuario no haya ingresado una cadena vacía.

💡 **Consejo del Mentor:** Recuerde que los objetos `String` en Java son inmutables: cada modificación crea un nuevo objeto en la memoria Heap. Para operaciones de modificación frecuente (como actualizar la máscara del juego turno a turno), la clase `StringBuilder` es inmensamente más eficiente en el uso de memoria.

📌 **Paso Siguiente:** Con los datos del jugador debidamente capturados y las variables de estado inicializadas, se procede a construir la lógica principal del bucle de juego.

4. Fase 3: Bucle del Juego y Manipulación Dinámica con StringBuilder (Unidades 1.3 y 1.4)

El núcleo de la aplicación reside en el bucle principal de juego, el cual se ejecuta en turnos sucesivos utilizando estructuras `while` o `for` mientras los `intentosRestantes` sean mayores a cero y la condición de victoria no se haya alcanzado.

Uso Obligatorio de `StringBuilder`

Debido a que las cadenas `String` son inmutables en Java, la modificación frecuente del texto durante la partida genera sobrecostos en memoria. Por ello, el estudiante debe utilizar obligatoriamente la clase `StringBuilder` para:

1. **Construir y actualizar la máscara oculta:** Crear una secuencia dinámica de guiones bajos y espacios que represente la palabra oculta (ejemplo de estado intermedio: `"J _ V _"`), reemplazando los guiones por las letras adivinadas en la posición exacta.
2. **Historial dinámico:** Mantener un registro acumulativo de las letras que el jugador ha arriesgado durante la sesión.

Lógica de Validación y Flujo de Control

- **Extracción y Comparación:** Extraer caracteres usando `.charAt()` y evaluar coincidencias mediante el uso de `.equalsIgnoreCase()` para garantizar insensibilidad a mayúsculas y minúsculas.
- **Manejo de Flujo:** Emplear la instrucción `break` para salir inmediatamente del bucle en caso de victoria temprana o rendición. Utilizar `continue` para saltar el procesamiento del turno si el usuario ingresa una letra repetida o un carácter inválido.
- **Retroalimentación Dinámica:** Implementar el operador ternario con la estructura conceptual `condicionAcierto ? "Mensaje de Éxito" : "Mensaje de Error"` para generar avisos instantáneos de respuesta en consola tras cada turno.

Lista de Chequeo de Validación

- [ ] Verificar que la entrada ingresada por el usuario no esté vacía antes de procesarla.
- [ ] Confirmar si la letra ingresada ya existe dentro del historial dinámico mantenido por el `StringBuilder`.
- [ ] Extraer el carácter relevante de la entrada utilizando `.charAt(0)` de forma segura.
- [ ] Evaluar las coincidencias entre la letra arriesgada y la palabra secreta empleando `.equalsIgnoreCase()`.
- [ ] Modificar el `StringBuilder` de la máscara en las posiciones correctas si la letra es acertada.
- [ ] Decrementar el contador `intentosRestantes` únicamente cuando la letra no pertenezca a la palabra.
- [ ] Aplicar `break` para finalizar la iteración si la máscara ya no contiene guiones ocultos o si se agotan los intentos.

📌 **Paso Siguiente:** Al finalizar el bucle principal de la partida y evaluar el resultado del jugador, el sistema debe dirigir el flujo hacia la generación y formateo del reporte final.

5. Fase 4: Resumen de Partida y Formateo de Resultados (Unidades 1.1 y 1.4)

Una vez concluida la partida (por victoria o por agotamiento de intentos), el programa debe calcular las métricas de desempeño y desplegar una pantalla final con el resumen detallado.

Requerimientos de Formateo y Cálculo Preciso

Toda la salida del reporte final debe ser procesada exclusivamente mediante el método `System.out.printf()`, aplicando los especificadores de formato correspondientes:

- `%s` para cadenas de texto (nombre del jugador, estado final).
- `%d` para valores enteros (intentos consumidos, intentos restantes).
- `%.2f` para valores numéricos de punto flotante formateados a dos decimales (porcentaje de precisión y puntaje final).

⚠️ **Prevención de Error Crítico en Cálculos:** Al calcular el porcentaje de aciertos o rendimiento (ej. dividiendo aciertos sobre total de intentos), recuerde que al operar entre dos variables de tipo `int`, Java realiza una **división entera** que descarta la parte decimal (dando como resultado `0` o `1`). Debe realizar la conversión explícita (*casting*) de al menos uno de los operandos a `double` (ej. `(double) aciertos / total * 100`) antes de operar para obtener la precisión decimal correcta.

💡 **Consejo del Mentor:** El uso de especificadores como `%-20s` o `%10d` dentro de `System.out.printf()` le permite definir el ancho de columna y la alineación del texto. Experimentos sencillos con estos modificadores le darán un acabado visual profesional a la consola.

Bloque de Salida Esperada en Consola

```
==================================================
            RESUMEN DE LA PARTIDA
==================================================
 Jugador:              CARLOS
 Estado Final:         ¡VICTORIA!
 Palabra Secreta:      JAVA
 Intentos Restantes:   4 / 6
 Porcentaje Aciertos:  83.33%
 Puntaje Final:        150.00 pts
==================================================
```

📌 **Paso Siguiente:** Una vez completada la presentación formateada de los resultados básicos, se presentan desafíos opcionales para extender la funcionalidad del programa hacia un nivel avanzado.

6. Fase 5: Retos de Expansión (Nivel Avanzado)

Para incentivar el aprendizaje autónomo y profundizar en el uso de métodos avanzados de cadenas y lógica matemática, se proponen los siguientes tres desafíos de extensión opcionales:

1. 💡 **Sistema de Pistas Inteligentes:** Implementar el uso de `.substring()` para revelar la primera letra sin descubrir o mostrar una pista contextual si al jugador le queda únicamente 1 intento restante.
2. ⚙️ **Niveles de Dificultad Dinámicos:** Ajustar automáticamente el valor de la constante `MAX_INTENTOS` en función de la longitud (`.length()`) de la palabra secreta seleccionada al inicio de la partida.
3. 🏆 **Multiplicador de Puntaje Matemático:** Calcular bonificaciones numéricas sobre el `puntaje` final aplicando fórmulas matemáticas que consideren la velocidad de resolución (turnos totales) y el número de fallos cometidos.

📌 **Paso Siguiente:** Para garantizar una implementación robusta y libre de fallos durante el desarrollo de estos requerimientos, es fundamental revisar la guía de depuración y prevención de errores en IntelliJ IDEA.

7. Fase 6: Puntos Críticos de Depuración (Debugging en IntelliJ IDEA)

Durante el desarrollo de aplicaciones en Java, es común encontrar comportamientos inesperados asociados a la gestión de memoria y flujo de consola. La siguiente matriz destaca los errores más frecuentes y cómo prevenirlos en el entorno IntelliJ IDEA.

Matriz de Depuración

Error Común de Programación	Causa Raíz en Java	Cómo Detectarlo y Evitarlo en IntelliJ IDEA
Comparación de cadenas mediante `**==**`	El operador `==` compara la referencia de memoria entre objetos y no su contenido textual.	Establecer un *breakpoint* en la línea de evaluación, inspeccionar las variables con la herramienta de depuración de IntelliJ IDEA y reemplazar la lógica por `.equalsIgnoreCase()` o `.equals()`.
Bucles infinitos en la navegación del menú	Lectura incorrecta del búfer al mezclar lecturas numéricas y de	Monitorear la variable de opción en la pestaña *Variables* del depurador; asegurar el parseo correcto mediante

	cadenas con la clase `Scanner`.	`Integer.parseInt()` sobre la lectura completa de línea.
Excepción `**StringIndexOutOfBoundsException**`	Intento de acceder a un índice inexistente mediante `.charAt()` o `.substring()` al procesar cadenas vacías.	Evaluar la longitud de la cadena con `.length() > 0` antes de invocar métodos de extracción, y utilizar la consola de *Watches* en el depurador para validar el tamaño del texto.

Mensaje de Cierre

Con esta guía de especificaciones, requerimientos de arquitectura y herramientas de prevención de errores a su disposición, le invitamos a iniciar la construcción de la aplicación en IntelliJ IDEA. Aplique cada concepto de las Unidades 1.1 a 1.4 con disciplina, rigurosidad técnica y pensamiento lógico. ¡Éxitos en el desarrollo!